写一下二分查找

标准二分查找（JS）

场景： 有序数组中查找"是否存在某个值"，命中返回索引，否则返回 `-1`。

```
/**
 * 标准二分查找：精确查找 target
 * @param {number[]} arr 已排序（升序）的数组
 * @param {number} target 目标值
 * @returns {number} 命中返回索引，否则 -1
 */
function binarySearch(arr, target) {
  let left = 0
  let right = arr.length - 1

  while (left <= right) {
    const mid = Math.floor((left + right) / 2)
    const midVal = arr[mid]

    if (midVal === target) return mid
    if (midVal < target) left = mid + 1
    else right = mid - 1
  }

  return -1
}
```

和项目里的二分查找做对比（区间命中 vs 精确命中）

你项目 src/renderer/src/utils/lyric/player.ts 里 `findCurrentLine(time)` 的目标不是"找到 `time === lyric.time` 的那一行"，而是：

- **区间命中：** 找到满足 `lyric[i].time <= time < lyric[i+1].time` 的 `i`（即"最后一个 ≤ time 的位置"）。

因此它和标准二分的关键差异是：

1. **命中条件不同**
   - 标准二分：`arr[mid] === target` 才算命中。
   - 项目版：只要 `time` 落在 `[midTime, nextTime)` 就命中并返回 `mid`。
2. **边界处理更业务化**
   - 项目版在进入循环前就处理：
     - `time < 第一行时间` → 返回 `0`（前奏阶段也高亮第一句）
     - `time >= 最后一行时间` → 返回最后一行索引（尾奏停在最后一句）

3. **循环结构相同，但“推进左右边界”的依据仍是比较 midTime**
    - `time < midTime` → 往左半区找（`right = mid - 1`）
    - 否则 → 往右半区找（`left = mid + 1`）

一句话总结：**标准二分解决“点查找”，项目二分解决“区间归属”。**

1. 动画控制：`requestAnimationFrame` (rAF) 是什么？为何优于 `setInterval`？

- rAF 是浏览器专门为动画提供的 API。它的回调执行时机**与屏幕刷新率同步**（通常 16.7ms / 60Hz），在事件循环（Event Loop）的**渲染阶段之前**执行，不会由于主线程阻塞造成掉帧。
- 另外，**当页面不可见或处于后台标签页时，rAF 会自动暂停**，这极大节省了 CPU 和电池资源，而 `setInterval` 仍在盲跑。

2. DOM 优化：`DocumentFragment` 的原理？

- `DocumentFragment`**（文档片段）：** 是一个**轻量级的、存在于内存中的真实 DOM 节点**。它独立于主文档树之外，因此对其进行内部 DOM 的增删改操作都不会触发页面的重排（Reflow）和重绘（Repaint）。只有在最后通过 `appendChild` 等方法将其一次性挂载到页面时，才只触发一次完整的渲染流水线。 const fragment = document.createDocumentFragment() this.container.appendChild(fragment)

3. `Map` 和普通对象 `{}` 的区别？为什么双语歌词匹配用 Map？

- **键的类型：** 对象的键只能是 `string` 或 `Symbol`，Map 的键可以是任意类型（包括数字、对象等）。歌词时间是数字，用 Map 不需要做隐式类型转换。
- **性能：** Map 在频繁增删查改的场景下性能更优，底层是哈希表实现，`has/get` 的平均时间复杂度是 O(1)。
- **有序性：** Map 保持插入顺序，对象在数字键时会自动排序。
- **大小获取：** Map 有 `.size` 属性，对象需要 `Object.keys().length`。

4. 色彩空间：为什么在提取颜色后，要转为 HSL 进行过滤而不是直接用 RGB？

- RGB 是红绿蓝发光模型，它是给显示器看的，不符合人类的感官直觉。
- **HSL（色相、饱和度、亮度）更贴近人眼认知。** 项目中需要剔除太暗、太白、太灰的颜色，直接在 HSL 维度的 `S`（饱和度）和 `L`（亮度）上设阈值（如排除 `<0.2` 或 `>0.8`）非常精准直观。
- **RGB → HSL 转换思路：** 将 R、G、B 归一化到 [0,1]，先由 max/min 求亮度 **L = (max+min)/2**，再由 max-min 差值求饱和度 **S**，最后根据最大分量是 R/G/B 分三种情况算色相 **H**（0-360°）。核心是从"亮不亮"→"鲜不鲜"→"什么颜色"逐步推导。

5. Canvas 绘图为什么要放在 rAF 里？不放会怎样？

- Canvas 的 `drawImage`、`toDataURL` 等操作是同步阻塞主线程的。如果直接在 `watch` 回调中执行，同步代码会立即执行，可能在一帧内堆积多次重复绘制（比如多个响应式数据连续变化），造成掉帧。
- 放在 `requestAnimationFrame` 里有两个好处：① 多次触发会被**合并到同一帧**执行（rAF 回调每帧只执行一次），天然起到节流作用；② 绘图时机与浏览器渲染节奏对齐，确保画面在下一帧完整呈现，避免撕裂。

6. 全局音频控制器的软件设计模式：这算是单例模式吗？除了挂载全局对象，前端还有什么方式实现单例？

- 算。全局仅暴露及维护唯一一个 Audio 控制器，保障多组件统一状态。
- 如果不使用 `window` 挂载，现代前端常使用 **ES Module 的导出**（模块在第一次加载被解析执行后结果会被缓存，导出的实例天然是单例），或使用状态库工具如 Pinia、Redux 维护单实例状态。

7. JS 特性应用：方法劫持 (Monkey Patching) 的原理是什么？结合你使用 `originPause.call(audio)` 说明为什么不用直接调用？

- **方法劫持：** 是指在运行时将目标对象的原生方法替换为我们自己的包装函数（先加自定义逻辑再调用原方法）。
- **为什么不能 `audio.pause()`：** 因为 `audio.pause` 已经被我们覆盖成了自定义的淡出函数。如果在自定义函数内部再调 `audio.pause()`，就等于调用自己 → 自己又调自己 → 无限递归 → 调用栈溢出（Stack Overflow）。所以我们必须事先把原生方法备份到 `originPause` 变量里，在包装函数内部调用这个备份。
- **为什么要用 `call`：** `originPause` 只是一个独立的函数引用，已经脱离了 `audio` 对象。直接 `originPause()` 调用时，`this` 会指向 `undefined`（严格模式）或 `window`，而浏览器原生的 `pause()` 方法内部需要 `this` 指向正确的 `HTMLAudioElement` 才能工作。所以必须用 `originPause.call(audio)` 手动把 `this` 绑回 audio 元素。
- **Monkey Patching 的优缺点：** 优点是对调用方完全透明，不需要改任何外部代码；缺点是侵入性强，调试困难（断点会进到包装函数），且如果多个库都 patch 同一个方法会互相覆盖。
- **更优雅的替代方案——Proxy：** ES6 的 `Proxy` 可以通过 `handler.get`/`handler.apply` 等 trap 拦截对象操作，不修改原始对象。但 `Proxy` 无法直接代理 DOM 原生对象的方法调用（浏览器内部方法要求正确的 `this`），所以本项目中 Monkey Patching 是更务实的选择。

8. 异步方案：音量淡入淡出的 `transitionVolume` 返回 Promise，如何控制 resolve 时机？

- 在 `new Promise` 的执行器中开启 `setInterval` 逐步修改音量。只有当音量达到目标值并执行了真正的物理播放/暂停后，才在该条件分支内手动调用 `resolve()`。外部 `await transitionVolume(...)` 会**挂起当前 async 函数**（不阻塞主线程），直到 `resolve` 被调用才继续往下执行。

9. Vue 底层：将内部 Vue 的响应式参数（如 `isPlay` ref）暴露到 Window 上，响应性会丢失吗？为什么？

- 不会丢失。Vue 3 的响应式核心是利用 **Proxy** 或提供内部 `getter/setter` 拦截的对象（RefImpl）。`ref` 生成的响应式对象本质还是那个被拦截的对象引用，不管被转移传递或赋值到哪个层级（包括 window），它自身的拦截器机制都在。
- **Vue 3 Proxy 响应性原理：** `reactive()` 内部用 `new Proxy(target, handler)` 包裹原始对象，在 `handler.get` 中做**依赖收集**（track，将当前正在运行的 effect/computed/watch 记录到该属性的订阅者集合），在 `handler.set` 中做**派发更新**（trigger，通知所有订阅者重新执行）。`ref()` 则是封装了一个带 `.value` 的 `getter/setter` 对象（RefImpl），访问 `.value` 时 track，修改 `.value` 时 trigger。

10. `setInterval` 的缺陷是什么？

- **不精确：** `setInterval(fn, 50)` 不是保证每 50ms 执行一次，而是每 50ms 将回调**加入宏任务队列**。如果主线程被阻塞，实际执行间隔会更长。
- **堆积问题：** 如果回调执行时间超过间隔时间，多个回调会在队列中堆积，导致连续快速执行。
- **替代方案：** 用递归 `setTimeout` 可以保证每次回调执行完后再等待固定时间；动画场景用 `requestAnimationFrame`。

11. `defineExpose` 的作用？Vue 3 的 `<script setup>` 为什么默认封闭？

- `<script setup>` 是 Vue 3 的编译时语法糖，默认**不对外暴露任何内部变量和方法**，组件实例上拿不到。这是为了更好的封装性和 Tree-shaking（未暴露的变量可以被编译器优化）。
- `defineExpose({ ... })` 显式声明哪些属性/方法可被父组件通过 `ref` 访问。项目中 `MusicPlayer` 通过它暴露 `audio` 元素和控制方法，使得 `App.vue` 能拿到并挂载到 `window.$audio`。

12. 浏览器机制对比：你项目里的 `transform: translateY(100%)` 和常见的 `display: none` / `v-if` 对引擎的区别？

- `v-if`：从组件到 DOM 节点，全部物理彻底销毁和析构。
- `v-show` / `display: none`：元素存在于 DOM 树，但从渲染树（Render Tree）剔除，不占空间，切换会引起**重排（Reflow）**。
- `transform` / `opacity: 0`：不仅存在 DOM 树也存在渲染树，保留原来的占位，改变它们仅在 GPU 层进行矩阵变换，即只触发**合成（Composite）**，性能最优。
- **为什么不用 `v-show`：** `v-show` 底层是 `display: none`，虽然不销毁组件但切换时会触发重排。而 `transform: translateY(100%)` 只触发合成，并且元素仍然在渲染树中，切回来时不需要重新构建渲染树，体验更流畅。

13. TypeScript：`type` 和 `interface` 的区别？

- **`interface`：** 专门用于描述对象的形状，支持**声明合并**（同名 interface 自动合并）和 `extends` 继承，适合定义公共 API 和第三方库的类型扩展。
- **`type`：** 更通用，除了对象还能定义联合类型、交叉类型、元组、映射类型等。不支持声明合并，但功能更灵活。
- **项目经验：** 一般组件 props 和 API 响应数据用 `interface`（方便扩展），工具函数的参数和联合类型用 `type`。

14. TypeScript：泛型怎么用？

- 泛型是**类型的参数化**，让函数/类/接口可以处理多种类型而不丢失类型信息。
- 常见用法：`function identity<T>(arg: T): T { return arg }`，调用时 `identity<string>('hello')` 或让 TS 自动推导。
- 项目中如请求函数 `request<T>(url): Promise<T>`，泛型让返回值类型随调用时指定而变化，既复用了逻辑又保持了类型安全。

15. Electron：简单介绍下什么是 Electron？主进程和渲染进程的区别？通信方式？

- **Electron 简介：** 一个使用 Web 技术（JS/HTML/CSS + 前端框架）开发跨平台桌面应用的框架。其核心原理是将 **Chromium**（提供浏览器渲染端）与 **Node.js**（提供底层系统级和文件级别的 API 接口）结合打包在了一起。
- **主进程（Main）：** 每个应用只有一个，运行 Node.js 环境，负责创建窗口（`BrowserWindow`）、管理系统级操作（菜单、托盘、文件系统等）。
- **渲染进程（Renderer）：** 每个窗口一个，运行 Chromium 环境，负责页面渲染。默认不能直接访问 Node.js API（安全沙箱）。
- **IPC 通信：** 通过 `ipcMain`（主进程监听）和 `ipcRenderer`（渲染进程发送）进行消息传递。现代做法是在 `preload.js` 中通过 `contextBridge.exposeInMainWorld` 安全地暴露 API 给渲染进程。
- **为什么桌面端更在乎 CPU：** 不像浏览器标签页被系统自动限制资源，Electron 应用是独立进程，后台高 CPU 占用会直接导致用户风扇狂转、电池耗尽，用户感知很强。

16. HTTP：歌曲封面的跨域问题怎么解决的？Canvas 里 `drawImage` 跨域会怎样？

- **Canvas 跨域污染：** 如果 `drawImage` 绑定了一张跨域图片且没有设置 CORS，Canvas 会变成"被污染"状态，调用 `toDataURL`/`toBlob`/`getImageData` 会直接报安全错误。
- **解决方式：** 给 `<img>` 标签设置 `crossOrigin = "anonymous"`，同时服务端响应头需包含 `Access-Control-Allow-Origin`。Electron 环境中由于主进程可以代理请求或直接操控网络策略，跨域问题相对容易处理。

17. Vite：热更新（HMR）原理？和 Webpack 的区别？

- **Vite HMR 原理：** 基于原生 ES Module。开发时不打包，浏览器直接请求源文件，Vite 的 dev server 按需编译。文件修改时，通过 WebSocket 通知浏览器只重新请求变化的模块，实现毫秒级热更新。
- **Webpack HMR：** 需要先把所有模块打包成 bundle，修改后重新编译整个依赖链（虽然有增量编译优化但还是慢），通过 `HotModuleReplacementPlugin` 替换模块。
- **核心区别：** Vite 冷启动快（不打包），HMR 快（只更新单个模块）；Webpack 冷启动慢（要打包），但生态成熟、配置灵活。Vite 生产环境用 Rollup 打包，Webpack 用自己的打包器。